



国防科技大学

NATIONAL UNIVERSITY OF DEFENSE TECHNOLOGY

高级软件工程 软件开发实践

姓 名: 王博伟 23026099, 黄成波 23026091, 司建新
23026154

任课教师: 毛新军, 王涛

目录

1 简介	3
2 系统实现	3
2.1 整体架构设计	3
2.2 微服务管理	4
2.3 通信设计	4
2.4 数据管理	5
2.4.1 PostgreSQL 关系型数据库	5
2.4.2 Neo4j 图数据库	7
2.4.3 Milvus 向量数据库	8
2.5 系统部署	9
3 系统测试	10

一 简介

本项目使用深度学习方法构建了针对历史文本的问答系统。该方法提供了一个用户交互界面，使用者可以利用该界面创建事件，将历史文字材料输入到系统中，系统将会使用先进的自然语言处理方法对文本进行处理，自动形成结构化形式；在用户提供了足够多事件信息后，系统将会使用一个以检索增强生成模型结构为基础的问答系统，针对这些信息进行检索和问答。本文通过提示学习方法在低资源场景下准确高效的实现了对命名实体识别、文本总结等多种下游任务的处理，避免了针对服务器资源 and 高质量数据集的依赖。同时，本项目针对检索增强生成模型的缺陷进行创新性改进，引入多轮检索问答过程，从而在解决用户提问时可以多阶段引入关系型数据、图数据、向量数据等多种类型信息，提高对于复杂问题的回答准确率。本系统针对交互界面和问答系统均进行了全面的测试，实验结果表明问答系统能够检索到与用户提问相符合的事件信息并由此生成正确的答案，在面对复杂问题时能够合理的将问题分为多个逻辑推理步骤并引入合适的外部数据进行解决。

二 系统实现

2.1 整体架构设计

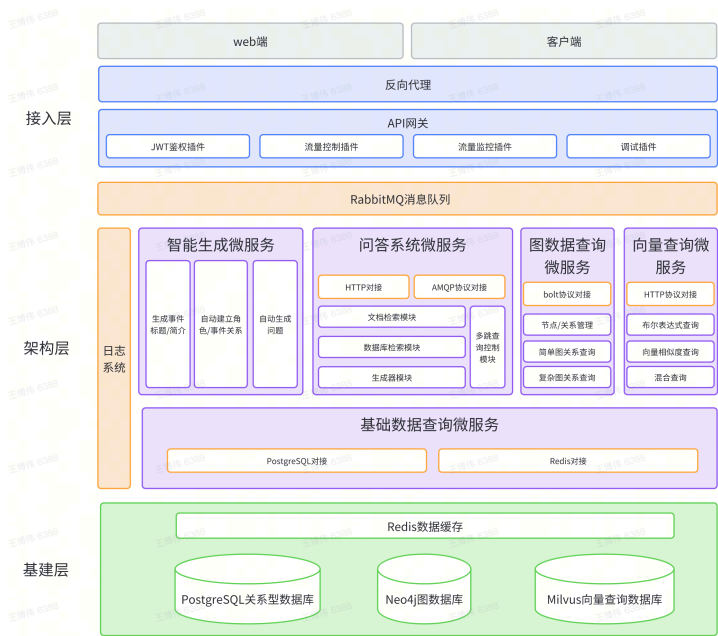


图 1 服务端整体架构图

服务端整体架构如 图 1 所示，共分为三个部分，分别为基建层、架构层与接入层：

- 基建层：该层是整个服务端应用的基础，负责提供数据存储的基础设施，包括 PostgreSQL 关系型数据库、Neo4j 图数据库，Redis 缓存数据库以及 Milvus 向量数据库；
- 架构层：该层是服务端应用的主体部分，负责接收用户的 API 请求，从基建层中获取数据并进行处理，最终将合适的数据返回客户端用于展示；
- 接入层：该层是服务端应用和客户端应用的对接部分，用于接收客户端请求并进行鉴权、限流、记录等处理，最终将请求转发到客户端对应的微服务中；

2.2 微服务管理

本系统整体采取了微服务架构设计，将系统划分为若干微服务，每个微服务模块独立的向外提供服务。本系统主要划分为了两个微服务，分别为核心微服务和人工智能微服务：

核心微服务是客户端应用的主要部分，负责维持整个系统的基础运行。通过核心微服务，用户可以进行注册账户、创建项目等操作，在成功创建项目后，即可在项目中进行操作；

人工智能微服务为系统提供自然语言处理相关的服务，包括事件摘要自动生成、自动关联事件参与角色等辅助服务，以及提供结构化事件问答系统服务。通过人工智能微服务，用户可以使用事件建模自动化相关功能，并使用结构化事件问答系统实现问答；

这两部分微服务建立在各类数据管理服务之上，包括关系型数据查询、图数据查询、向量数据查询以及缓存管理等。

2.3 通信设计

根据微服务设计思想，每个微服务都有独立的数据管理设施，不同微服务之间的数据尽可能不要互通，以免形成强依赖关系；但微服务之间仍然会存在必要程度的信息交流，如事件触发通知等，因此需要对微服务之间的通信关系及方式进行设计。

系统采用了 RabbitMQ 消息队列作为微服务通信基础设施。RabbitMQ 是一个基于 AMQP 协议（高级消息队列协议）的消息队列实现，具有可伸缩性以及消息持久化等众多优点。RabbitMQ 的使用主要基于 exchange 和 queue 两个概念，其中：

- exchange 为发送方终端，消息发送者可以指定一个键和一个信息，将其交给某个指定的 exchange，该 exchange 将会按照其定义的路由规则和键来将消息进行转发；
- queue 则为接收方终端，每一个 queue 都有一个指定名称，消息接收者可以通过指定 queue 的名称来监听某一个或某些 queue，当 exchange 将消息路由到这些 queue 中时，消息接收者即可接收到信息。

系统采用了 RabbitMQ 提供的 topic（话题）消息模式，在这种方式中，消息发送者在发送消息时将会指定这个消息会发送到哪些 topic 上，而接收者则会绑定某一个或某一类 topic，如果发送者指定的 topic 与接收者指定的 topic 的模式相匹配，则接收者即可接收到该消息。这类消息模式的好处在于 topic 不仅可以是一个精确的键，

还可以是一类模式，允许使用通配符，如 `*.rabbit.*` 等。利用该种消息模式，可最大程度的实现消息的精准发布与接收，同时具有灵活性与准确性的特点。

消息标识	消息名称	含义
<code>content_create</code>	数据创建消息	事件、角色、场景及历史背景数据创建
<code>content_update</code>	数据更新消息	事件、角色、场景及历史背景数据更新
<code>content_remove</code>	数据移除消息	事件、角色、场景及历史背景数据被移除
<code>content_done</code>	数据完成消息	事件、角色、场景及历史背景数据完成状态更改

图 2 系统通信 topic 设计表

系统主要定义的 topic 如 图 2 所示，其中每一类 topic 下均有四个子 topic，分别为 `event`、`chara`、`scene` 和 `worldview`，分别代表该消息是事件、角色、场景还是历史背景数据。这些消息将会在核心微服务的数据管理 API 被调用时发出，接收方分别为：

- 图数据管理服务：当图数据管理服务接收到实体创建、更新或移除消息时，将会将该数据对应的节点数据进行相应的处理；
- 向量数据管理服务：当数据的完成状态发生变化时，对其嵌入向量做相应处理：如果事件从未完成状态变成已完成状态，则将数据存入到 `done` 分区，并对其进行文本嵌入，获取到嵌入向量并进行存储；如果事件从已完成状态变成未完成状态，则将数据移动到 `undone` 分区，且将已有的嵌入向量删除；
- 人工智能微服务：当数据的状态发生变化时，将会立即更新 `redis` 缓存，保证从缓存中得到的数据是最新的，避免出现数据不一致的情况；

2.4 数据管理

2.4.1 PostgreSQL 关系型数据库

系统采用了 PostgreSQL 数据库作为基础数据库来存储关系型数据。PostgreSQL 数据库是一款优秀的关系型数据库，遵循对象-关系管理系统的设计原则，可以使用 SQL 语句进行查询。为了简化对于数据库的查询，系统采用了名为 Prisma 的 ORM (Object-Relational Mapping, 对象-关系映射) 框架。利用该 ORM 框架，可以在 typescript 语言中实现强类型的数据库操作。

数据库主要的数据表如 图 3，图 4，图 5 所示，该表用于存储事件信息以及各类实体信息。

编号	字段名	字段类型	字段说明
1	id	SERIAL	数据唯一标识, 自动增加
2	serial	INTEGER	事件序号
3	name	TEXT	事件名称
4	description	TEXT	事件描述
5	createdAt	TIMESTAMP (3)	创建时间
6	updatedAt	TIMESTAMP (3)	更新时间
7	deleted	TIMESTAMP (3)	删除时间
8	unit	INTEGER	时间单位
9	start	TIMESTAMP (3)	起始时间
10	end	TIMESTAMP (3)	结束时间
11	type	"EventType"	事件类型
12	done	BOOLEAN	事件是否已完成
13	projectId	INTEGER	所属项目 id

图 3 事件数据表 EVENT 结构

编号	字段名	字段类型	字段说明
1	id	SERIAL	数据唯一标识, 自动增加
2	name	TEXT	角色名称
3	alias	TEXT[]	角色别名
4	description	TEXT	角色描述
5	avatar	TEXT	角色头像
6	deleted	TIMESTAMP (3)	删除时间
7	unit	INTEGER	时间单位
8	start	TIMESTAMP (3)	起始时间
9	end	TIMESTAMP (3)	结束时间
10	done	BOOLEAN	角色信息是否已完成
11	projectId	INTEGER	所属项目 id

图 4 角色数据表 CHARA 结构

编号	字段名	字段类型	字段说明
1	id	SERIAL	数据唯一标识, 自动增加
2	name	TEXT	场景名称
3	alias	TEXT[]	场景别名
4	description	TEXT	场景描述
5	deleted	TIMESTAMP (3)	删除时间
6	done	BOOLEAN	场景信息是否已完成
7	projectId	INTEGER	所属项目 id

图 5 场景数据表 SCENE 结构

2.4.2 Neo4j 图数据库

系统采用了 Neo4j 图数据库存储图数据。Neo4j 是一款图数据库, 属于 NoSQL (非关系型数据库) 的范畴。这种数据库基于图论的概念, 以节点和节点间关系的形式来组织数据, 每个节点和关系都具有标签, 且可以附属一些其他属性。这种特殊的数据组织形式在针对特定场景的需求时能够发挥很大的作用, 如分析用户关系数据时, 如果使用普通的关系型数据库来记录用户关系, 则需要使用外键等方法, 需要进行多次递归查询, 效率很低; 而使用图数据库则可以直接的将这些关系表达出来, 查询时只需要指定关系标签以及查询深度即可进行查询, 数据库对这类关系查询进行了很大的优化, 因此性能相对更好。

在系统中, Neo4j 作为辅助数据库, 协助基础数据库存储事件和实体之间的关系信息。通过 Neo4j 数据库, 系统可以以类似于事件知识图谱的方式对事件和实体的关系进行存储; 与真正的事件知识图谱不同的地方在于, 此数据库中存储的节点只包含类别和 id 等必要信息, 本身并不存储详细信息, 若用户需要在查询关系后获得节点的详细信息则需要在基础数据库中进行查询。这样的分离设计保证了职责的专一性, 既保证了对于特定种类信息的查询效率, 又避免了数据过度冗余。数据表结构设计如图 6 所示。

编号	字段名	字段说明	1
LABEL	节点对应数据的类别，包括 event、chara、scene 和 worldview	2	id
节点对应数据的 id	3	projectId	节点对应数据所属项目的 id
4	name	节点对应数据的标题	5
order	节点对应数据的排序依据，用于批量查询节点时进行排序		

图 6 节点数据结构

2.4.3 Milvus 向量数据库

系统采用了 Milvus 数据库来存储向量数据。Milvus 是一款高性能的向量数据库，这种数据库原生支持存储向量类型，且内建了许多向量查找算法，从而能够快速实现对输入向量最近邻查找。在系统中，使用该数据库存储事件等描述信息的嵌入向量，从而实现对这些内容的语义检索。这种数据库的基本形式与普通的关系型数据库基本一致，但存在一些为优化向量查询而设计的特性：

- 在该数据库中，数据的基础组织形式为 collection（集合），对应关系型数据库中的 table（表）；
- 每一个 collection 中必定包含至少一个向量字段，该字段将会被作为向量检索的依据；在创建完 collection 后，必须为向量字段指定 index（索引），索引决定了对此向量应该采用何种查找算法；在指定了一种类别的 index 后，数据库会把之后添加的数据都按照该 index 指定的数据形式进行处理；
- 在每一个 collection 中，还可以区分出若干 partition（分区），每一个分区代表一个 collection 中的一部分数据组成的集合，这些数据会在硬盘存储中被组织到同一个位置。用户在进行向量检索时可指定某几个分区进行检索，这时其他分区的向量将不会纳入搜索范围，从而提高了检索的效率；

在系统中共有四类数据具有描述信息，分别为事件、角色、场景与背景信息，因此系统在 Milvus 中建立了对应的四类 collection；在每一类 collection 中，系统为向量字段建立了 HNSW index，从而可以对数据执行 HNSW 算法查找；此外，在每一个 collection 中均有 done 和 undone 两类 partition，含义为：

- done：表明该数据的描述信息已经完成，用户不会对描述信息进行修改，此时将会使用文本嵌入算法将文本转为嵌入向量并存入数据库中；这些数据将会作为可索引的数据，应用于文本检索和问答系统中；

- **undone**：表明该数据的描述信息还没有完全完成，用户可能会对信息进行再次修改，此时不应对该数据进行文本嵌入，所以此类数据的向量字段将使用与嵌入向量同维度的 0 向量来占位；这些数据将不会被索引到；

2.5 系统部署

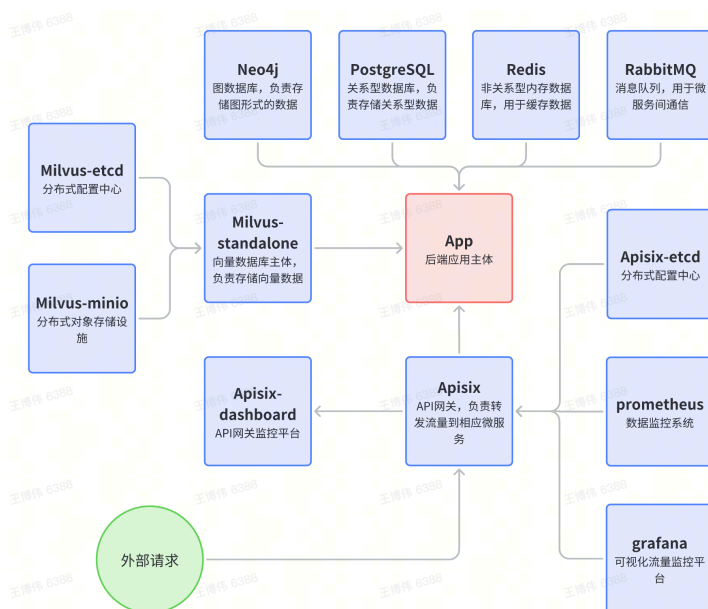


图 7 系统部署依赖关系图

本课题主要采用了云原生相关技术和方法来完成系统部署任务。

在容器化方面，系统采用了 **Docker** 作为容器平台进行了容器部署，并使用 **docker-compose** 技术将服务端所有需要的依赖组合在一起，从而实现统一关系。容器的分类主要分为如下部分：

- **App**：该部分为服务端应用主体，负责向客户端提供服务；
- **存储和通信设施**：该部分为服务端应用到的所有存储和通信设施，包括 **PostgreSQL**、**Neo4j**、**Redis**、**Milvus** 以及 **RabbitMQ**；
- **Apisix 网关管理**：该部分为 **Apisix** 相关的容器，负责接收流量并进行流量管理、监控和转发。

容器之间具有一定的依赖关系，一些容器需要在某些指定容器开始运行后才能够正常启动，系统部署依赖关系如 图 7 所示。

在持续集成方面，系统采用了 **Github Actions** 来实现 **CI/CD** 功能，当代码通过 **git** 工具推送到 **Github** 代码管理网站时，将会自动触发 **Github Actions**，从而在远程部署服务器中重新构建 **docker** 镜像并重启 **docker compose** 容器组，实现线上服务的及时更新。

三 系统测试

本项目部署于腾讯云服务器上，首先输入网页链接打开本系统，如 图 8 所示。该页面为本系统的初始页面，即工作台，工作台上可以放置若干组件；

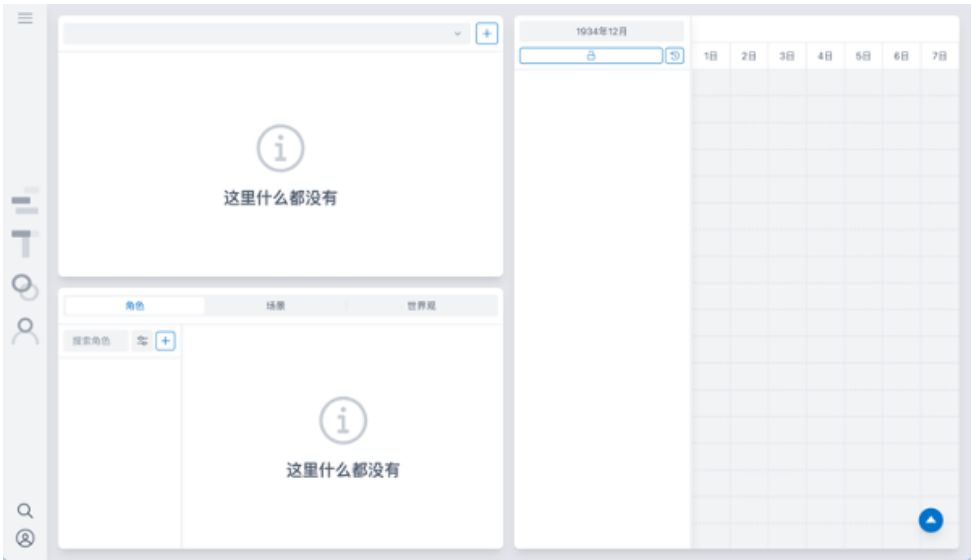


图 8 系统初始界面

左侧为侧边栏，可以通过拖拽图标的方式对工作台进行重新布局，如 图 9 所示。

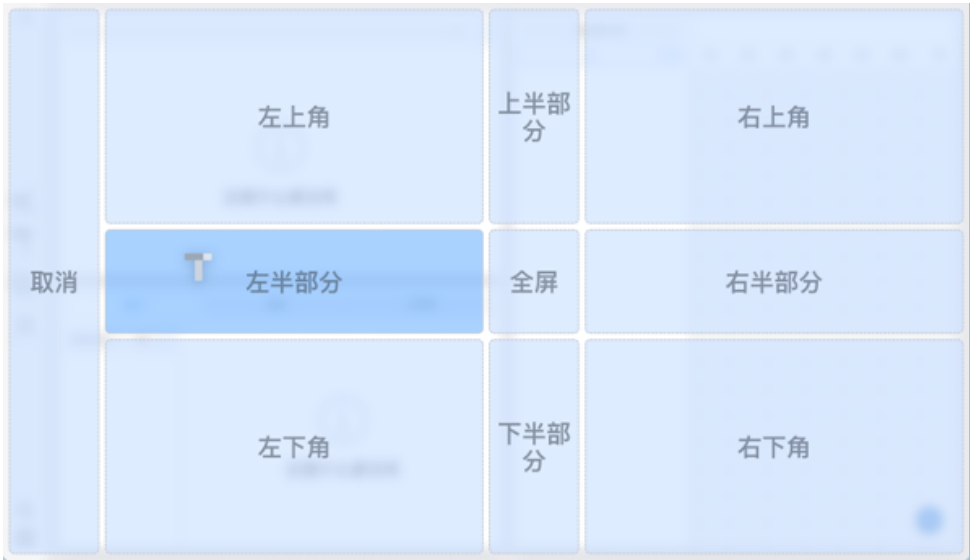


图 9 通过拖拽图标进行重新布局

在工作台中，可以在多个组件中创建事件，此处使用时序图组件进行测试，如 图 10 所示。



图 10 创建事件

创建事件完成后，点击事件条目，即可在编辑器组件中编辑事件信息，如 图 11 所示。此处以四渡赤水战役的历史信息为例。

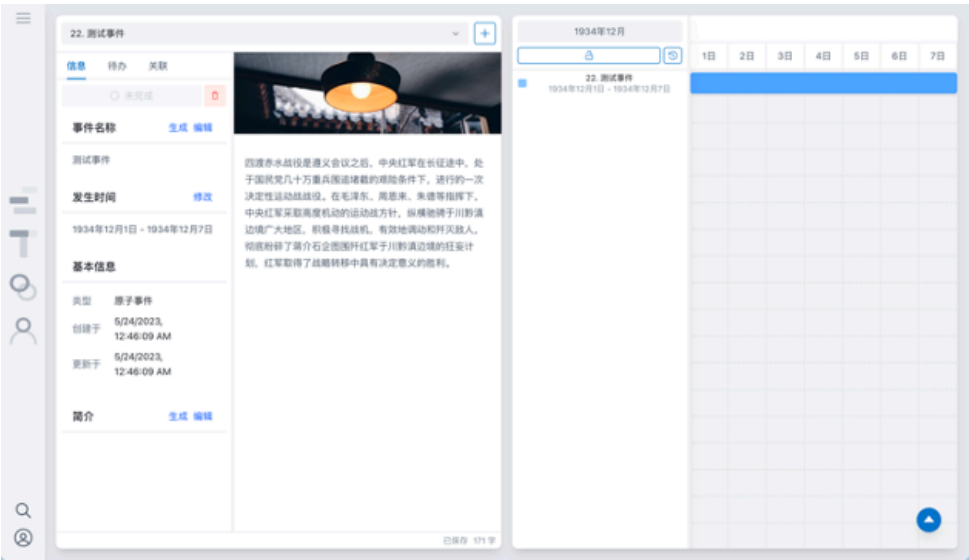


图 11 编辑事件详细信息

完成事件信息后，即可利用人工智能技术自动生成事件名称、事件简介以及参与角色等信息，如 图 12 所示。

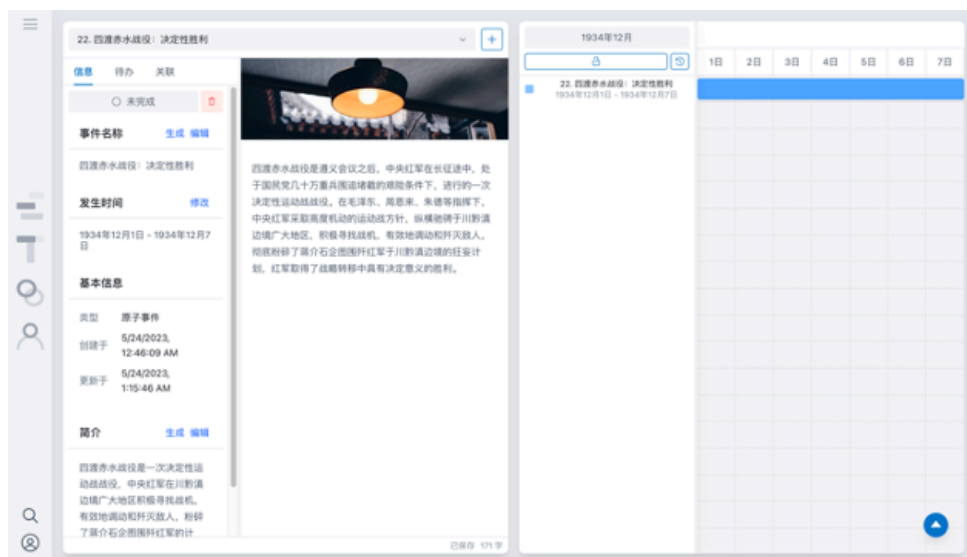


图 12 自动生成事件标题和简介

在事件各种信息完成后，可点击“未完成”按钮，将事件状态转为“已完成”，如图 13 所示。此时，该事件的信息将被用于问答系统中。

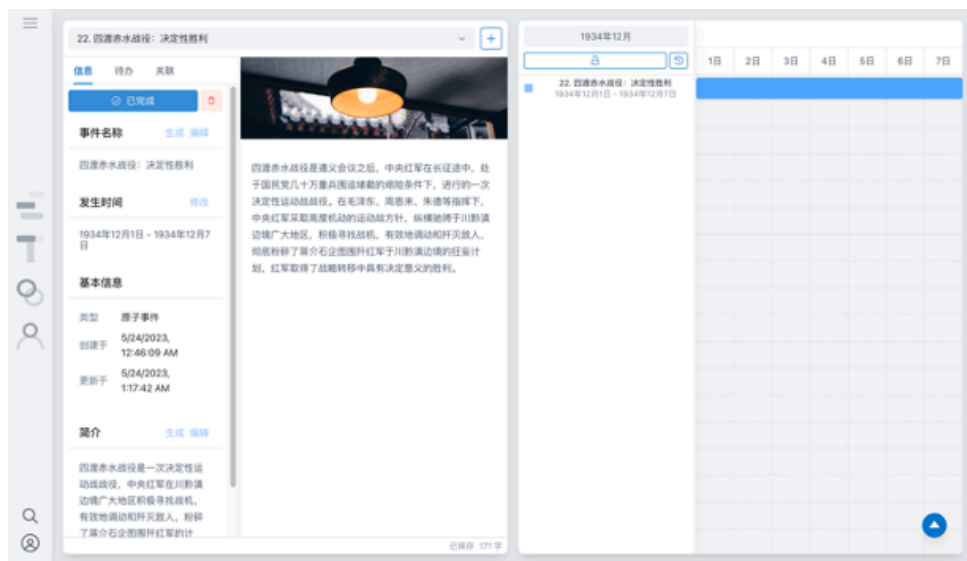


图 13 将事件状态转为已完成

为了对问答系统进行测试，先创建三个事件信息，分别为抗日战争、遵义会议和四渡赤水战役，如图 14 所示。这三个事件的关系为：抗日战争为其余两个事件的父事件；遵义会议导致了四渡赤水战役的胜利。

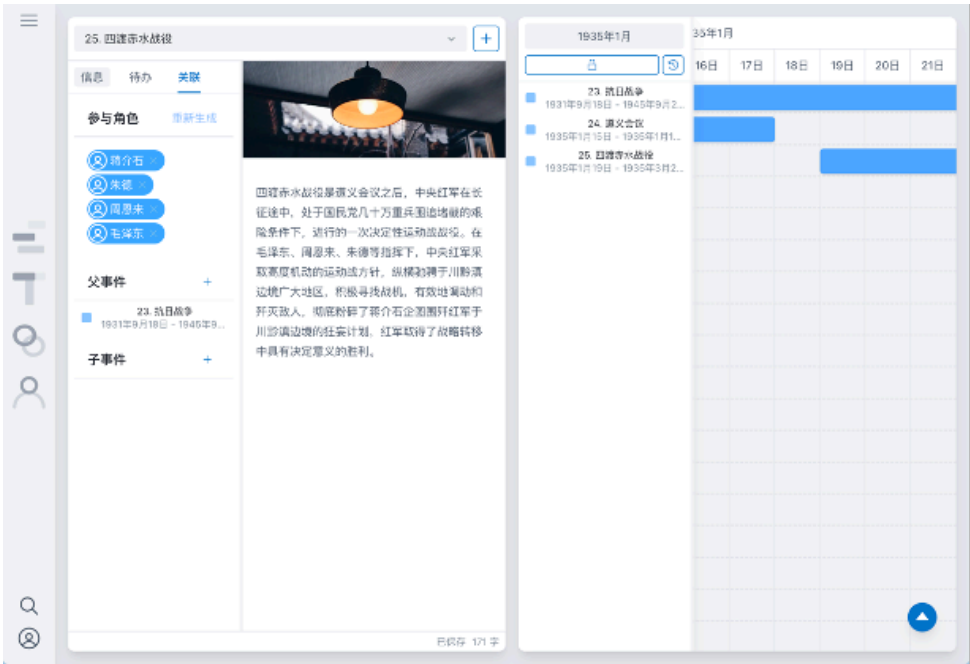


图 14 创建测试事件

打开问答系统交互界面，系统会根据已完成事件来提供若干样例问题，如 图 15 所示。



图 15 系统提供样例问题

系统经过处理后，将会给出问题的答案，如 图 16 所示。在给出答案的同时，系统也会给出答案的依据，用户点击答案中的蓝色链接即可了解该答案所依据的事件。



图 16 系统给出问题答案

当询问具有更复杂逻辑关系的问题时，系统将会使用多轮检索生成步骤以追求更合理的答案，如 图 17 所示。

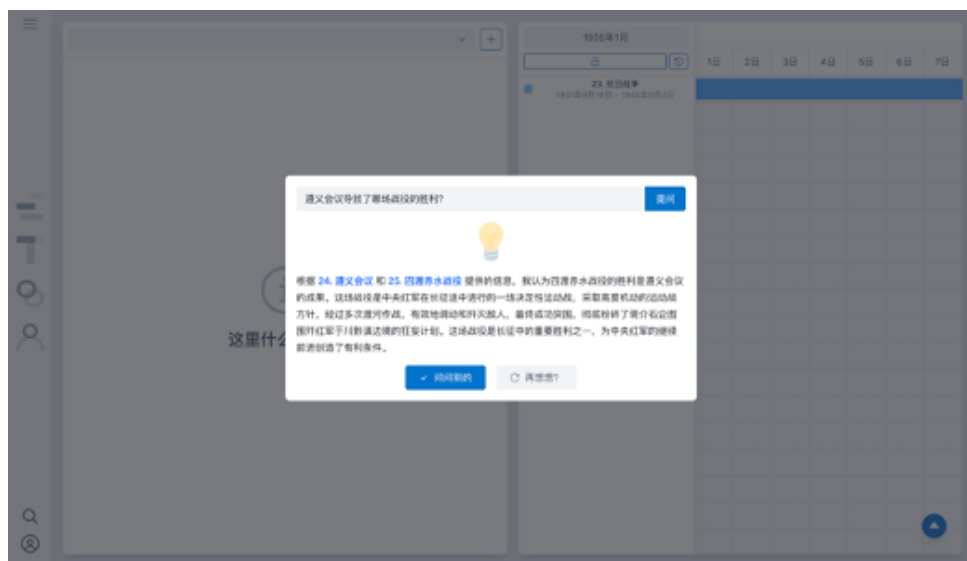


图 17 系统给出复杂问题答案